

torch.func.stack_module_state#

https://pytorch.org/docs/stable/generated/torch.func.stack_module_state.html

Description:

Prepares a list of `torch.nn.Modules` for ensembling with `vmap()`. Given a list of `M` `nn.Modules` of the same class, returns two dictionaries that stack all of their parameters and buffers together, indexed by name. The stacked parameters are optimizable (i.e. they are new leaf nodes in the autograd history that are unrelated to the original parameters and can be passed directly to an optimizer). Here's an example of how to ensemble over a very simple model: When there's submodules, this follows state dict naming conventions

Function Signature

```
torch.func.stack_module_state(models) → params,  
buffers[source]#
```

Return type

Returns:

tuple[dict[str, Any], dict[str, Any]]

Code Examples

```
num_models = 5
batch_size = 64
in_features, out_features = 3, 3
models = [torch.nn.Linear(in_features, out_features) for i in
range(num_models)]
data = torch.randn(batch_size, 3)

def wrapper(params, buffers, data):
    return torch.func.functional_call(models[0], (params,
buffers), data)

params, buffers = stack_module_state(models)
output = vmap(wrapper, (0, 0, None))(params, buffers, data)

assert output.shape == (num_models, batch_size, out_features)
```

```
import torch.nn as nn

class Foo(nn.Module):
    def __init__(self, in_features, out_features):
        super().__init__()
        hidden = 4
        self.l1 = nn.Linear(in_features, hidden)
        self.l2 = nn.Linear(hidden, out_features)

    def forward(self, x):
        return self.l2(self.l1(x))

num_models = 5
in_features, out_features = 3, 3
models = [Foo(in_features, out_features) for i in
range(num_models)]
params, buffers = stack_module_state(models)
print(list(params.keys())) # "l1.weight", "l1.bias",
"l2.weight", "l2.bias"
```

Notes & Warnings

WARNING

Warning All of the modules being stacked together must be the same (except for the values of their parameters/buffers). For example, they should be in the same mode (training vs eval).

Detailed Documentation

Documentation

Prepares a list of `torch.nn.Modules` for ensembling with `vmap()`.

Given a list of `M` `nn.Modules` of the same class, returns two dictionaries that stack all of their parameters and buffers together, indexed by name. The stacked parameters are optimizable (i.e. they are new leaf nodes in the autograd history that are unrelated to the original parameters and can be passed directly to an optimizer).

Here's an example of how to ensemble over a very simple model:

When there's submodules, this follows state dict naming conventions

All of the modules being stacked together must be the same (except for the values of their parameters/buffers). For example, they should be in the same mode (training vs eval).

```
tuple[dict[str, Any], dict[str, Any]]
```